



ANGULAR 6

Lesson 08



The background of the slide features an abstract design with a dark blue field and a large, flowing red shape that enters from the left and bottom. On the right side, there is a red L-shaped graphic element.

HTTP REQUESTS / OBSERVABLES

Objectives

Objectives

- HTTP Requests
- Sending GET Requests
- Sending a PUT Request
- Transform Responses with Observable Operators (`map()`)
- Using the Returned Data
- Catching Http Errors
- `Pipe()`, `map()`, `catchError()`
- Interceptors
- Basics of Observables & Promises
- Analysing a Built-in Angular Observable
- Building & Using a Custom Observable
- Understanding Observable Operators
- Using Subjects to pass and listen to data

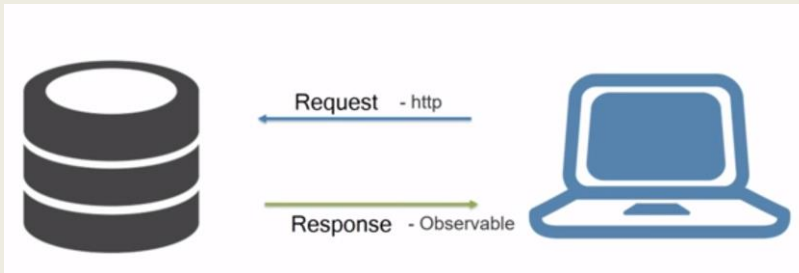


HTTP REQUESTS

An abstract graphic featuring a dark blue background with several red shapes. On the left, there is a large, irregular red shape. To its right, a smaller red shape is partially visible. On the right side of the image, there is a red L-shaped line that extends horizontally and then vertically upwards.

HTTP Requests

- Angular applications often obtain data using http
- Application issues http get requests to a web server which returns http response-Observable to the application.
- Application then processes that data



Most front-end applications need to communicate with a server over the HTTP protocol, in order to download or upload data and access other back-end services.

Angular provides a simplified client HTTP API for Angular applications

Sending GET Requests

- When you want to fetch data from server, you use get method from HttpClient.
- Get method will return the body of response as an untyped JSON Object by default.
- The server's data API determines the shape of JSON data.
- Asynchronous method will send an HTTP request and will return an Observable which will emits the requested data when the response is received.
- The get() method takes 2 arguments – the endpoint URL from which to fetch, and an options Object which is used to configure the request.



When you want to fetch the data from server, we achieve by using the HttpClient API.

It has a method names as get() which will have 2 parameters. Those types are 1. URL to hit the endpoint and receive the data from server 2. options Object which is used to mention the details of request types and its metadata.

Return type of this method is Obervable which will emit the requested data when response is received.

Sending a PUT Request

- When you want to update data from server, you use `put()` method from `HttpClient`.
- Put method will replace the resource with updated data in the server.
- As for any of the HTTP methods that return an observable, `update()` method must `subscribe()` to the observable returned from the `HttpClient.put()` in order to initiate the request.
- The `put()` method takes 2 arguments – the endpoint URL from which to fetch, and an options Object which is used to configure the request.



When you want to fetch the data from server, we achieve by using the `HttpClient` API.

It has a method names as `get()` which will have 2 parameters. Those types are 1. URL to hit the endpoint and receive the data from server 2. options Object which is used to mention the details of request types and its metadata.

Return type of this method is `Observable` which will emit the requested data when response is received.

`update()` must `subscribe()` to the observable returned from the `HttpClient.put()` in order to initiate the request.

Add instructor notes here.

Introducing RxJs

- RxJs stands for Reactive Extensions for Javascript, and its an implementation of Javascript.
- It is a ReactiveX library for JavaScript.
- It provides an API for asynchronous programming with observable streams.
- ReactiveX is a combination of the best ideas from the Observer pattern, the Iterator pattern, and functional programming.
- Observable is a RxJS API. Observable is a representation of any set of values over time. All angular Http methods return instance of Observable. Find some of its operators.
- map: It applies a function to each value emitted by source Observable and returns final Observable.
- catch: It is called when an error is occurred. catch also returns Observable.



The RxJS library is quite large.

It's up to us to add the operators we need

// Add map operator

<https://cdnjs.cloudflare.com/ajax/libs/rxjs/4.1.0/rx.map>

// Add all operators to Observable

<https://cdnjs.cloudflare.com/ajax/libs/rxjs/4.1.0/rx.all.js>

// Add map operator

import 'rxjs/add/operator/map';

// Add all operators to Observable

import 'rxjs/Rx';

Transform Responses with Observable Operators

- Operators are functions that build on the observables foundation to enable sophisticated manipulation of collections.
- For example, RxJS defines operators such as [map\(\)](#), [filter\(\)](#), [concat\(\)](#), and [flatMap\(\)](#).
- Operators take configuration options, and they return a function that takes a source observable.
- When executing this returned function, the operator observes the source observable's emitted values, transforms them, and returns a new observable of those transformed values.



Our web application responds Asynchronously. It clearly states that, we can request many things at a time to our web application.

All requests will be handled once the result is available.

What we have is RxJS library, which will allow us to work in asynchronous data programming.

This library have some utility functions which will help us to Convert an existing code for async operations into Observables.

Using this Observable will be able to fetch back the results.

There are n number of oerators available in RxJS libraray which is used to filter or fetch the data as expected.

Using the Returned Data



- Data which is received from server using HTTPClient API, can be fetched and displayed to client by following below steps:

1. Import the Service
2. Inject the Service in the Component using Angular's Dependency Injection.
3. In the Components `ngOnInit()` method, get the RxJs Observable to the list of data's and using `subscribe()` method , subscribe to it.
4. Store all the fetched list of data on the component so that It can be rendered in the template.



Example for Fetching data from Server

```
import { BookService } from "../book/book.service";
//@Component decorator goes here

export class BookListComponent implements OnInit {
  books: Book[];

  // (2) Inject
  constructor(private bookService: BookService) {
    this.books = [];
  }

  ngOnInit() {
    // (3) Subscribe
    this.bookService.list().subscribe((books: Book[]) => {

      // (4) Store
      this.books = books;
    });
  }
}
```

Create a new component named BookListComponent which Implements BookService.

Declare an array of Book type.

Inject the created BookService in the constructor using Angular's Dependency Injection.

BookService will hit the server and will fetch back the datas from the server.

Inside ngOnInit life cycle method invoke subscribe method to subscribe the data received from Server using the RxJs Observable.

You will not be able to receive the data from Observable till you dont subscribe to it.

Pass that subscribed data into array and through the template you can pass all the datas to Angular view page.

Transform Responses with Observable Operators



■ Map operator

```
import { map } from 'rxjs/operators';  
const nums = of(1, 2, 3);  
const squareValues = map((val: number) => val * val);  
const squaredNums = squareValues(nums);  
squaredNums.subscribe(x => console.log(x));  
  
// Logs  
// 1  
// 4  
// 9
```



Imported map and of operator from RxJS library.

Of – any number of provided values in Sequence.

Of can emit an Object, array and function.

Map operator - *it passes each source value through a transformation function to get corresponding output values.*

RxJS Utility functions

- RxJS -> Reactive Extensions for Javascript
- It is a library for reactive programming Using observables that makes it easier do compose asynchronous or callback-based code.
- RxJS provides an implementation of the observable type which is needed until the type becomes part of the language and until browser supports it.
- When you want to work with observables this library provides utility functions for creating and working with it



RxJS Library Utility Functions



Utility functions will help to



filter the stream



compose multiple stream



map value to different type



Iterate through the values in a stream and finally



convert existing code for asynchronous operation into observables

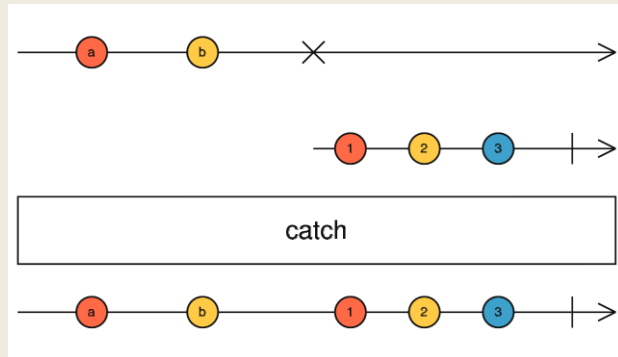
Catching Http Errors

- In addition to the `error()` handler, RxJS provides the `catchError` operator that lets you handle known errors in the observable recipe.
- For instance,
 - *suppose you have an observable that makes an API request and maps to the response from the server.*
 - *If the server returns an error or the value doesn't exist, an error is produced.*
 - *If you catch this error and supply a default value, your stream continues to process values rather than erroring out.*



catchError

- When you catch the error and supply a default value, your stream continues to process values rather than erroring out



Catching Http Errors

- Here's an example of using the catchError operator

```
import { ajax } from 'rxjs/ajax';
import { map, catchError } from 'rxjs/operators';
// Return "response" from the API. If an error happens, // return an empty array.
const apiData = ajax('/api/data').pipe(
  map(res => {
    if (!res.response) {
      throw new Error("Value expected!");
    }
    return res.response;
  }),
  catchError(err => of([]))
);
apiData.subscribe({
  next(x) { console.log('data: ', x); },
  error(err) { console.log('errors already caught... will not run'); }
});
```

CatchError - Catches errors on the observable to be handled by returning a new observable or throwing an error.

catchError has a parameter of type selector and its return type is operatorFunction.

catchError Function:

catchError<T, O extends ObservableInput<any>>(selector: (err: any, caught: Observable<T>) => O): OperatorFunction<T, T | ObservedValueOf<O>>

selector : a function that takes as arguments err, which is the error, and caught, which is the source observable, in case you'd like to "retry" that observable by returning it again.

OperatorFunction<T, T | ObservedValueOf<O>>: An observable that originates from either the source or the observable returned by the catch selector function.



Interceptors

- Interceptors provide a way to Intercept the HTTP requests and responses to transform or handle them before passing them along.
- It provides a mechanism to intercept and/or mutate the incoming responses and outgoing requests.
- You can achieve this using the HTTPInterceptor Interface API in @angular/common/http.
- ```
interface HttpInterceptor {
 intercept(req: HttpRequest<any>, next: HttpHandler):
 Observable<HttpEvent<any>>
}
```
- You can use this Interceptor for features like logging and caching.



Interceptors are used to intercept the outgoing request and incoming response. It is nothing but a filter. When you want to filter the response which we are receiving from server and when we want to filter the request going to server, we use HTTPInterceptor.

It can be achieved using the interface HTTPInterceptor API in @angular/common/http.

This interface has a method intercept() which we will override in created class by implementing this Interface.

- You can use multiple interceptors in one application. Angular will follow the order of multiple interceptor way you provide it.
- For Example: You have provided interceptors as first A then B and finally C.
- Request will be Intercepted in order as A then B and then C.
- And responses will be intercepted first with C then B then finally with A.
- If you want to enable or disable interceptor dynamically, that can be build inside the interceptor itself.



<https://stackblitz.com/github/melcor76/interceptors?file=src%2Fapp%2Finterceptors%2Ffake.interceptor.ts>

<https://github.com/melcor76/interceptors/tree/master/src/app/interceptors>

Add instructor notes  
here.

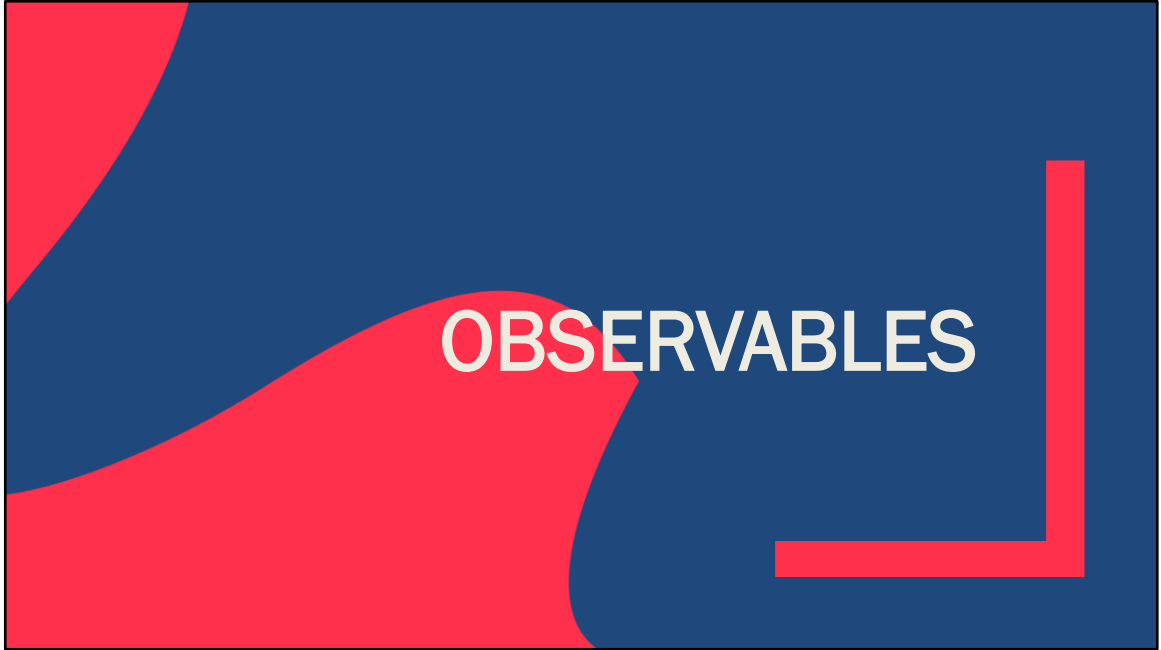
# Demo

- HttpGet
- HttpGetfromJSONFile
- HttpGetErrorHandling

Cap



Add the notes here.



Add instructor notes here.

## Basics of Observables & Promises

- Observables provide support for passing messages between publishers and subscribers in an application.
- Observables offer significant benefits over other techniques for event handling, asynchronous programming, and handling multiple values.
- Observables are declarative—that is, you define a function for publishing values, but it is not executed until a consumer subscribes to it. The subscribed consumer then receives values as the function completes, or until they unsubscribe.

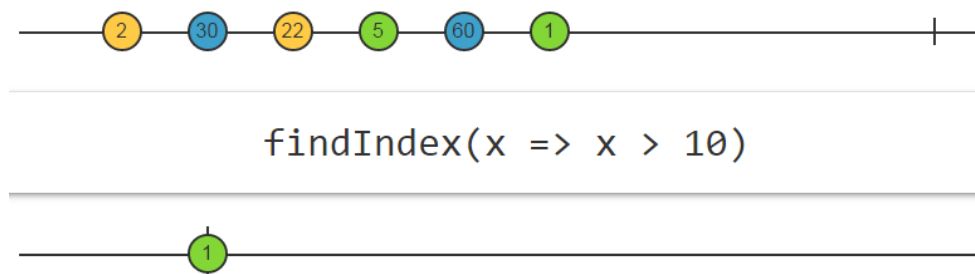


Data sequences can take many forms such as a stream of data from a backend web service or a set of system notifications or a series of events such as user input.

Reactive extensions represent a data sequence as an observable sequence, commonly just called an observable.

A method can be subscribed to an observable to receive asynchronous notifications as new data arrives. The method can then react with the arrived data. The method is notified when there is no more data or one or more errors occur. Since an observable works like an array we can use the `map` operator.

We can visualize observable sequences with interactive diagrams from



Marble diagram

Add instructor notes here.

## Basics of Observables & Promises

- An observable can deliver multiple values of any type—literals, messages, or events, the context. The API for receiving values is the same whether the values are delivered synchronously or asynchronously. Because setup and teardown logic are both handled by the observable, application code only needs to worry about subscribing to consume values, and when unsubscribing. Whether the stream was keystrokes, an HTTP response, or an interval, the interface for listening to values and stopping listening is the same.
- Because of these advantages, observables are used extensively within Angular, and are recommended for app development as well.

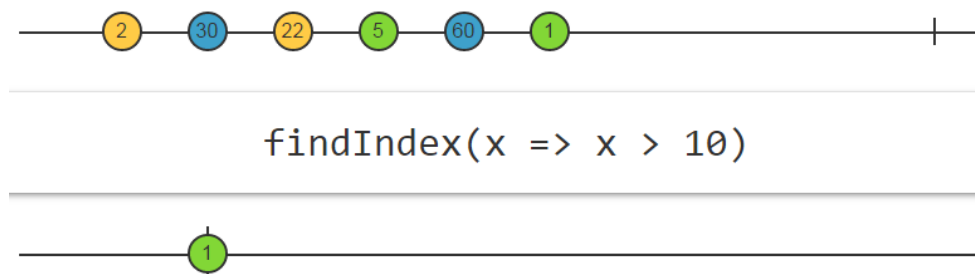


Data sequences can take many forms such as a stream of data from a backend web service or a set of system notifications or a series of events such as user input.

Reactive extensions represent a data sequence as an observable sequence, commonly just called an observable.

A method can be subscribed to an observable to receive asynchronous notifications as new data arrives. The method can then react with the arrived data. The method is notified when there is no more data or one or more errors occur. Since an observable works like an array we can use the `map` operator.

We can visualize observable sequences with interactive diagrams from



Marble diagram

Add instructor notes here.

## Basics of Observables & Promises

- Observables is a part of ReactiveX library also known as rxjs
  - `import { Observable } from 'rxjs/Observable';`
- Observables is like an array whose items arrived asynchronously. The role of ReactiveX is to make asynchronous programming
- Observables help to manage asynchronous data, such as data coming from a backend web service or a set of system notifications or a series of events such as user input
- Observables work with multiple values
- Observables are cancellable
- Observables use JavaScript functions such as map, filter & reduce

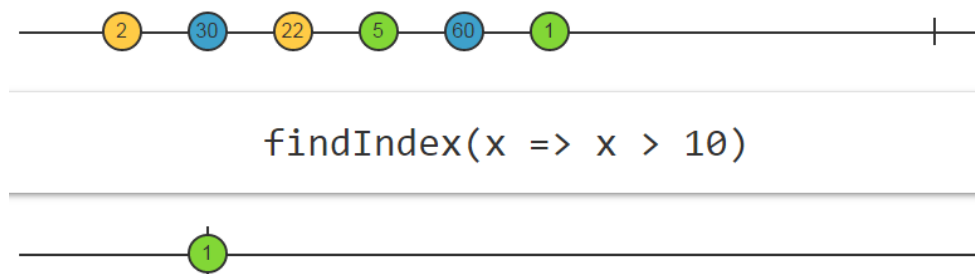


Data sequences can take many forms such as a stream of data from a backend web service or a set of system notifications or a series of events such as user input.

Reactive extensions represent a data sequence as an observable sequence, commonly just called an observable.

A method can be subscribed to an observable to receive asynchronous notifications as new data arrives. The method can then react with the arrived data. The method is notified when there is no more data or an error occurs. Since an observable works like an array, we can use the `map` operator.

We can visualize observable sequences with interactive diagrams from



Marble diagram



## Analysing a Built-in Angular Observable

- Angular makes use of observables as an interface to handle a variety of common asynchronous operations. For example:
- You can define custom events that send observable output data from a child to a parent component.
- The HTTP module uses observables to handle AJAX requests and responses.
- The Router and Forms modules use observables to listen for and respond to user-input events.



- Consider you are going to send the data from child component to parent component.
- In this case the parent component will have a function That accepts the data emitted from the child component.
- This parent component will get the data before calling the `ngOnInit()` method which is a lifecycle hook method.
- `@Output` is the annotation and `EventEmitter` class is used to interact between the parent and the child component.
- Both the above are available in `@angular/core` package.



- Event emitter is a class which is used to emit the custom events and it is used only with the @Output decorator.
- This class extends the subject class which is used to apply the provider subscriber kind of communication .
- A subject is like an observable but can multicast to many observers
- Whereas plain observables are unicast that is each subscribed observer owns an Independent execution of the observable.



## Building & Using a Custom Observable

- Use the Observable constructor to create an observable stream of any type.
- The constructor takes as its argument the subscriber function to run when the observable's subscribe() method executes.
- A subscriber function receives an Observer object, and can publish values to the observer's next() method.



# Building & Using a Custom Observable

## ■ Create observable with constructor

```
// This function runs when subscribe() is called
function sequenceSubscriber(observer) {
 // synchronously deliver 1, 2, and 3, then complete
 observer.next(1);
 observer.next(2);
 observer.next(3);
 observer.complete();
 // unsubscribe function doesn't need to do anything in this
 // because values are delivered synchronously
 return {unsubscribe() {};}
}
// Create a new Observable that will deliver the above sequence
const sequence = new Observable(sequenceSubscriber);
// execute the Observable and print the result of each notification
sequence.subscribe({
 next(num) { console.log(num); },
 complete() { console.log('Finished sequence'); }
});
// Logs: 1 2 3 Finished sequence
```

Use the Observable constructor to create an observable stream of any type.

The constructor takes as its argument the subscriber function to run when the observable's subscribe() method executes.

A subscriber function receives an Observer object, and can publish values to the observer's next() method.

next() method - handler for each delivered value. Called zero or more times after execution starts.

An Observable instance begins publishing values only when someone subscribes to it.

You subscribe by calling the subscribe() method of the instance, passing an observer object to receive the notifications.

Once all data is received, you invoke the complete() method to complete the



observer.

complete() - handler for the execution-complete notification. Delayed values can continue to be delivered to the next handler after execution is complete.

## Understanding Observable Operators

- Operators
- Operators are functions that build on the observables foundation to enable sophisticated manipulation of collections. For example, RxJS defines operators such as `map()`, `filter()`, `concat()`, and `flatMap()`.
- `map`:
  - Applies a given project function to each value emitted by the source Observable and emits the resulting values as an Observable.



## Map Operator Example:

```
import { map } from 'rxjs/operators';

const nums = of(1, 2, 3);

const squareValues = map((val: number) => val * val);

const squaredNums = squareValues(nums);

squaredNums.subscribe(x => console.log(x));
```



map is one of the simple operator. It takes a projection function and then applies it to each value that comes from the source observable.

Operators take configuration options, and they return a function that takes a source observable.

When executing this returned function, the operator observes the source observable's emitted values, transforms them, and returns a new observable of those transformed values which you are seeing above.



## of()

- Converts the arguments to an observable sequence Each argument becomes a next notification



## Example:

```
import { of } from 'rxjs';

of(10, 20, 30).subscribe(
 next => console.log('next:', next),
 err => console.log('error:', err),
 () => console.log('the end'),
);

// result:
// 'next: 10'
// 'next: 20'
// 'next: 30' argument becomes a next notification.
```

For example: `of(10, 20, 30)` creates an observable that will emit 10, 20, and 30, one right after another.

Using subscribe method you are receiving the values from returned Observable



## pipe()

- Unlike map, which is an operator, pipe is a method on Observable which is used for composing operators.
- You can use pipes to link operators together. Pipes let you combine multiple functions into a single function. The pipe() function takes as its arguments the functions you want to combine, and returns a new function that, when executed, runs the composed functions in sequence.
- `of(1,2,3).map(x => x + 1).filter(x => x > 2);`
- and turn it into this
- `of(1,2,3).pipe( map(x => x + 1), filter(x => x > 2) );`



Unlike map, which is an operator, pipe is a method on Observable which is used for composing operators.

You can use pipes to link operators together.

Pipes let you combine multiple functions into a single function.

The pipe() function takes as its arguments the functions you want to combine, and returns a new function that, when executed, runs the composed functions in sequence.

- pipe offers the following benefits:
- It cleans up Observable.prototype by removing operators
- It makes the RxJS library more tree-shakeable
- It makes it easier to write and use third-party operators (since you don't have to worry about patching Observable.prototype).



pipe offers the following benefits:

It cleans up Observable.prototype by removing operators

It makes the RxJS library more tree-shakeable

It makes it easier to write and use third-party operators (since you don't have to worry about patching Observable.prototype).

## Using Subjects to pass and listen to data

- Subject:
- *A Subject is like an Observable but can multicast to many Observers. Subjects are like EventEmitters: they maintain a registry of many listeners.*
- An RxJS Subject is a special type of Observable that allows values to be multicasted to many Observers. While plain Observables are unicast
- Subject - A subscriber will only get published values thereon-after the subscription is made



*A Subject is like an Observable but can multicast to many Observers.*

*Subjects are like EventEmitters: they maintain a registry of many listeners.*

An RxJS Subject is a special type of Observable that allows values to be multicasted to many Observers.

While plain Observables are unicast

Subject - A subscriber will only get published values thereon-after the subscription is made

- **Subject:** On subscribing it always gets the data which is pushed after it's subscription i.e. *previous pushed values are not received*.
- Internally to the Subject, subscribe does not invoke a new execution that delivers values. It simply registers the given Observer in a list of Observers
- Every Subject is an Observer. It is an object with the methods next(v), error(e), and complete().
- To feed a new value to the Subject, just call next(theValue), and it will be multicasted to the Observers registered to listen to the Subject



```
import { Subject } from 'rxjs';
const subject = new Subject<number>();
subject.subscribe({
 next: (v) => console.log(`observerA: ${v}`)
});
subject.subscribe({
 next: (v) => console.log(`observerB: ${v}`)
});
subject.next(1);
subject.next(2);
```

**OUTPUT::**

```
// observerA: 1
// observerB: 1
// observerA: 2
// observerB: 2
```

- We have two Observers attached to a Subject, and we feed some values to the Subject:



# Summary

## You have learned

- How to use HTTP Requests
- To Send GET Requests & PUT Request
- To Transform Responses with Observable Operators (`map()`)
- To catch Http Errors
- Basics of Observables & Promises
- To build & use a Custom Observable
- Observable Operators
- How to use Subjects to pass and listen to data



Summa